

MAS Final Project: Main Orchestration and CLI (`main.py`) Complete Code Documentation

1. Scope

This chapter documents `main.py` in full, including:

- CLI argument model,
- runtime mode dispatch,
- configuration/default resolution,
- interactive/simulation/evaluation loops,
- symmetry detection entrypoint,
- verification-refinement orchestration,
- logging and reproducibility behavior.

2. Imports and Dependencies

`main.py` imports:

- standard libs:
 - `argparse`
 - `logging`
 - `random`
 - `sys`
 - `Optional` from `typing`
- third-party:
 - `pygame`
- project modules:
 - `config` as `cfg`
 - `WarehouseEnv` from `env.py`
 - `CooperativePlanner` from `pathfinding.py`
 - `refine_planner_with_conflicts` from `refinement.py`
 - `verify_on_quotient` from `verification.py`

This file is the orchestration layer and does not implement planning or dynamics directly.

3. Seed Control (`set_seed`)

Function:

- `set_seed(seed: int) -> None`

Behavior:

- calls `random.seed(seed)`.

Purpose:

- reproducible randomized initialization (robot directions, random placements).

4. Interactive Runtime (`run_interactive`)

Function:

- `run_interactive(args: argparse.Namespace) -> None`

Flow:

1. Create render-enabled environment:
 - `env = WarehouseEnv(render=True)`
2. Create planner with horizon:
 - `CooperativePlanner(..., plan_horizon=args.plan_horizon)`
3. Enter infinite event/action/render loop:
 - process pygame events,
 - delegate to `env.handle_event(event)`,
 - exit on:
 - window close,
 - q key.
4. Compute planner actions:
 - `actions = planner.compute_actions(env)`
5. Step environment:
 - `_, _, done, _, _ = env.step(actions)`
6. If episode done:
 - `env.reset()`
7. Render frame:
 - `env.render()`
8. On keyboard interrupt:
 - `pygame.quit()`
 - `sys.exit(0)`

Characteristics:

- continuous loop intended for live visual inspection/debugging.

5. Batch Simulation (`run_simulation`)

Function:

- `run_simulation(args: argparse.Namespace) -> None`

Flow:

1. Build logger (`warehouse`).
2. Create environment with optional rendering:
 - `WarehouseEnv(render=args.render)`
3. Create planner with chosen horizon.
4. Initialize `total_collisions = 0`.

5. Loop `episode_idx` over `args.episodes`:
 - `env.reset()`
 - run steps until:
 - done, or
 - `args.steps_per_episode` reached.
6. If rendering enabled:
 - handle pygame quit and q key exits.
7. Each step:
 - `actions = planner.compute_actions(env)`
 - `env.step(actions)` returns `episode_collisions`
 - accumulate into `total_collisions`.
8. If render enabled, call `env.render()`.
9. Every `args.log_interval` episodes:
 - compute average collisions per agent per episode:
 - `avg = total_collisions / ((episode_idx + 1) * env.num_agents)` with zero guard.
 - log structured progress.

Use case:

- workload runs with optional visualization and periodic metrics.

6. Evaluation Mode (`run_eval`)

Function:

- `run_eval(args: argparse.Namespace) -> None`

Flow:

1. Create logger.
2. Create no-render environment.
3. Create planner.
4. Call `env.evaluate(...)` with:
 - `num_episodes=args.eval_episodes`
 - `max_steps_per_episode=args.steps_per_episode`
 - `progress_every=1`
 - `logger`
5. Log resulting average collision rate.

This mode delegates rollout details to env-level evaluation helper.

7. Verify-Refine Loop (`run_verify_refine`)

Function:

- `run_verify_refine(args, planner: Optional[CooperativePlanner] = None) -> CooperativePlanner`

Purpose:

- iterative safety hardening:
 - verify planner behavior on quotient states,
 - if unsafe, add constraints from conflicts,
 - repeat up to configured iterations.

Flow:

1. Create logger.
2. Create no-render environment.
3. If no planner provided:
 - create fresh planner.
4. Initialize `last_result = None`.
5. Loop `iteration` from 0 to `args.refine_iterations - 1`:
 - run `verify_on_quotient(...)` with:
 - `horizon/trials/include_shelves/min_separation/progress/logger`
 - store `last_result`.
6. If verification `safe=True`:
 - log pass + `delta_q`
 - call `_log_refine_summary(...)`
 - return planner immediately.
7. If unsafe:
 - log warning,
 - extract:
 - `conflicts`
 - `counterexample_trace`
 - call `refine_planner_with_conflicts(...)` with max constraint cap,
 - log refinement summary.
8. If loop exhausts without safety:
 - warn,
 - log final summary if available,
 - return planner.

Design note:

- planner is modified in-place by refinement through constraint injection.

8. Refinement Summary Logger (`_log_refine_summary`)

Function:

- `_log_refine_summary(logger, planner, result) -> None`

Outputs:

1. average collision rate from verification result.
2. flattened list of position constraints from planner constraint table.
3. flattened list of edge constraints from planner constraint table.
4. total counts and sample entries (up to 10 each) via logger.

Purpose:

- post-iteration visibility into learned safety constraints.

9. CLI Definition (`_build_parser`)

Function:

- `_build_parser()` -> `argparse.ArgumentParser`

Defines arguments:

- `--mode` (`interactive|simulate|eval`, default `interactive`)
- `--render` (flag)
- `--config` (default `config.yaml`)
- `--seed` (optional int)
- `--cell-size` (optional int)
- `--episodes` (simulate count, default 8)
- `--steps-per-episode` (default 200)
- `--log-interval` (default 1)
- `--eval-episodes` (default 3)
- `--plan-horizon` (optional int)
- `--detect-symmetry` (flag)
- `--verify-refine` (flag)
- `--verify-horizon` (optional int)
- `--verify-trials` (optional int)
- `--verify-include-shelves` (flag)
- `--verify-progress` (default 1)
- `--min-separation` (optional int)
- `--refine-iterations` (optional int)
- `--refine-max-constraints` (optional int)

This parser is the only external user interface for runtime behavior selection.

10. Default Application (`_apply_defaults`)

Function:

- `_apply_defaults(args)` -> `None`

For arguments left `None`, assigns defaults from `config.py`:

- `plan_horizon <- cfg.PLAN_HORIZON`
- `verify_horizon <- cfg.VERIFY_HORIZON`
- `verify_trials <- cfg.VERIFY_TRIALS`
- `min_separation <- cfg.MIN_SEPARATION`
- `refine_iterations <- cfg.REFINE_ITERATIONS`
- `refine_max_constraints <- cfg.REFINE_MAX_CONSTRAINTS`

This ensures CLI optional parameters inherit config-level values.

11. Program Entry (main)

Function:

- `main(argv=None) -> None`

Execution sequence:

1. Build parser and parse args.
2. Configure logging:
 - INFO level
 - timestamped format.
3. Attempt config load from `args.config`:
 - `cfg.load_from_yaml(...)`
 - warning log on failure.
4. Apply config defaults into args via `_apply_defaults`.
5. If `--seed` provided:
 - set RNG seed,
 - log seed.
6. If `--cell-size` provided:
 - override `cfg.CELL_SIZE`.

Branch routing:

1. If `--detect-symmetry`:
 - instantiate env + reset,
 - import `build_quotient_model`,
 - print orbits and quotient summary,
 - return.
2. Else if `--verify-refine`:
 - run verify-refine loop,
 - return.
3. Else route by `--mode`:
 - `interactive -> run_interactive`
 - `simulate -> run_simulation`
 - `eval -> run_eval`

Module guard:

- `if __name__ == "__main__": main()`

12. Execution Precedence and Override Semantics

Effective precedence:

1. Hardcoded defaults in `config.py`
2. YAML overrides from `--config`
3. CLI explicit flags/values

Example:

- if `config.yaml` sets `planning.horizon=40` and user passes `--plan-horizon 30`, planner uses 30.

13. Logging Behavior

Standard logger name:

- "warehouse"

Used in:

- simulation progress,
- evaluation result,
- verification pass/fail,
- refinement summaries,
- config load failures.

Output formatting includes timestamp + level.

14. Runtime Contracts with Other Modules

`main.py` expects:

- WarehouseEnv API:
 - constructor, `reset`, `step`, `render`, `evaluate`, `handle_event`.
- CooperativePlanner API:
 - constructor and `compute_actions`.
- verification API:
 - `verify_on_quotient(...)` result dict with `safe`, `delta_q`, etc.
- refinement API:
 - `refine_planner_with_conflicts(...)` returning summary dict.

It does not inspect module internals; it orchestrates via public interfaces.

15. Determinism and Reproducibility in Main

Seed handling:

- only Python `random` is explicitly seeded.

Implications:

- environment spawns and initial robot directions become reproducible under fixed seed/config.
- deterministic planner and environment logic then follow from deterministic inputs.

16. Error and Exit Behavior

1. Interactive mode:
 - exits on QUIT, key `q`, or `KeyboardInterrupt`.

2. Simulation mode:
 - returns early on render-window quit or key q.
3. Config load:
 - exceptions are logged as warnings, run continues.
4. Verify-refine:
 - returns planner even if full safety not reached by iteration cap.

17. Full Function Coverage Checklist

All functions in `main.py` documented:

- `set_seed`
- `run_interactive`
- `run_simulation`
- `run_eval`
- `run_verify_refine`
- `_log_refine_summary`
- `_build_parser`
- `_apply_defaults`
- `main`

18. Chapter Summary

`main.py` is the project control plane.

It maps CLI intent to concrete execution loops, ensures config/seed/default consistency, and coordinates the full verify-refine lifecycle that connects planning, symmetry-aware verification, and constraint-based safety improvement.